Design Document

Platinum 1 - Complex Shader [GitHub]

Contents

Splash Shader 2
- What it is 2
- What it is used for 2
- Why it works the way it works 2
  - Render Modes 2
  - Uniforms 2
  - Vertex Shader 3
  - Fragment Shader 4

Splatter Shader 5
- What it is 5
- What it is used for 5
- Why it works the way it works 5
  - Render Modes 5
  - Uniforms 5
  - Vertex Shader 6
  - Fragment Shader 6

Splash Shader [Github]

What it is

This shader is a spatial shader designed for use in 3D scenes, specifically to create a blood splash effect. It utilizes various rendering modes and uniforms to achieve a dynamic and visually appealing effect that can be applied to plane or quad meshes.

What it is used for

Within Fowl Play, this shader is used to enhance the visual feedback of combat interactions by simulating blood splashes when the player or enemies are hit.

Since the core of our game revolves around combat, and the theme is dark and gritty, having a somewhat realistic blood splash effect is essential to convey the impact of attacks.

Why it works the way it works

Render Modes

blend_mix: lets the blood splash blend smoothly with whatever's behind it, so it looks like it's really part of the scene instead of just sitting on top.

depth_draw_opaque: makes sure the splash appears at the right depth, so it doesn't show up in front of things it should be behind.

cull_back: skips drawing the back side of the splash (which you'd never see anyway), making things a bit faster.

unshaded: ignores lighting, so the splash always looks the same no matter how the lights in the scene change.

Uniforms

main_color: sets the main color of the blood splash so splashes don't all look the same. With instance we can actually make it unique across different splashes.

noise_texture: adds randomness to the splash, making it look more natural and less repetitive.

distortion_strength: controls how wobbly or distorted the splash appears, helping it feel more dynamic.

noise_power: adjusts how strong the noise effect is, letting you go from subtle to wild splashes. Higher makes it more paint like, lower makes it more like droplets.

base_alpha: changes how see-through the splash is, so you can make it more or less noticeable.

angle_influence: tweaks how much the splash's angle changes its look, adding extra variation.

gradient_strength: controls the strength of the gradient effect, allowing for more or less pronounced transitions.

Vertex Shader

mat4 billboard_matrix = VIEW_MATRIX creates a matrix that makes it look exactly like the camera is looking.

VIEW_MATRIX * INV_VIEW_MATRIX[0] is basically a way to ensure the billboarded object's orientation is purely dependent on the camera's rotation, without inheriting any rotation from the original object.

By setting ***billboard_matrix[0],[1],[2]*** this way, we're stripping away any rotation the original object had and forcing it to perfectly align with the camera's X, Y, and Z axes. This makes the object always face the camera flat-on, regardless of its own initial rotation.

billboard_matrix[3] = VIEW_MATRIX * MODEL_MATRIX[3] takes the camera's view, and then put our object's actual position (MODEL_MATRIX[3]) into it.

billboard_matrix[0][0] is essentially the scaling/rotation along the X-axis.

length(MODEL_MATRIX[0].xyz) calculates the length (how big it is) of the X-axis vector of the model.

billboard_matrix[0][0] = length(MODEL_MATRIX[0].xyz) forcefully sets the X-axis scale of the billboard matrix to match the size of the model along that axis. This ensures that the splash appears correctly sized in the 3D space.

The same is done with the y-axis(***billboard_matrix[1][1])*** and z-axis(***billboard_matrix[2][2]***), correspronding with ***length(MODEL_MATRIX[1].xyz)*** and ***length(MODEL_MATRIX[2].xyz)*** respectively.

MODELVIEW_MATRIX is the final calculation Godot uses to draw the object from the camera's perspective. So basically, we set the ***MODELVIEW_MATRIX*** to our billboard_matrix, and then use that for rendering.

INSTANCE_ID is a number that's unique to each copy of an object if there are many of them (in our case, a swarm of particles). We're essentially just taking that ID number and putting

it into the 'red' color channel of the object. By setting the other color channels (green and blue) to 0, we ensure that the color is primarily determined by the instance ID.

Fragment Shader

We first calculate the position and angle for each pixel.

vec2 center = vec2(0.5) sets the center of the texture.

vec2 dir = UV - center calculates the direction from the center of the current pixel's coordinate.

float radius = length(dir) * gradient_strength calculates the distance from the center to the current pixel. Using ***gradient_strength,*** we can control how quickly the gradient fades out from the center.

float angle = atan(dir.y, dir.x) / (PI * 2.0): atan calculates the angle of the pixel relative to the center, and we normalize it to a value between 0 and 1 by dividing by (PI * 2.0), because a full circle is 2 * PI radians.

float gradient = 1.0 - max(radius, angle * angle_influence) calculates a gradient value based on the radius and angle. The max function ensures that the gradient is influenced by both the distance from the center and the angle. This creates a gradient effect that fades out from the center.

float distortion_factor = COLOR.r * distortion_strength uses the red channel of the color (which is set to the instance ID). This means each individual instance (e.g., each particle) could have a slightly different amount of distortion.

vec2 distorted_uv = UV + vec2(distortion_factor) applies the distortion to the UV coordinates, creating a wavy effect based on the noise texture.

texture(noise_texture, distorted_uv).r samples the noise texture at the distorted UV coordinates, and gets the red channel value.

pow(..., noise_power) adjusts the contrast/brightness of the noise (red channel).
A noise_power > 1.0 makes dark areas darker, accentuating bright noise.
A noise_power < 1.0 makes dark areas brighter, smoothing out the noise.

vec3 final_color = main_color.rgb * (vec3(gradient) * vec3(texture_value))
The final color is determined by:

1. ***main_color***: The base color you choose for the effect.
2. ***gradient***: Our circular fade, making it brighter in the middle.

3. ***texture_value***: The noisy texture, adding a chaotic or organic look.

float alpha_threshold = 1.0 - (gradient * texture_value) calculates the alpha threshold, which determines how transparent the splash is based on the gradient and noise. This is used for ***'ALPHA_SCISSOR_THRESHOLD'***, which creates sharp cutouts.

ALBEDO = final_color sets the final color of the splash.

ALPHA = base_alpha sets the overall transparency of the splash.

ALPHA_SCISSOR_THRESHOLD = alpha_threshold sets the alpha threshold for the splash. This means that pixels with an ***'ALPHA'*** value below this threshold will not be drawn at all, effectively discarding them from rendering.

Splatter Shader [Github]

What it is

Like the splash shader, this shader is also a spatial shader designed for use in 3D scenes. However, it is specifically tailored to create a blood splatter effect that can be applied to the floor.

What it is used for

Within Fowl Play, this shader is used to enhance the visual feedback of combat interactions by simulating blood splatters on the ground when the player or enemies are hit.

Why it works the way it works

Render Modes

diffuse_lambert: applies a basic diffuse lighting model, so the splatter reacts to light in the scene.

specular_schlick_ggx: adds a specular highlight to the splatter, making it look shiny and wet, which is fitting for blood.

Uniforms

main_color: sets the main color of the blood splatter, allowing for variation in appearance.

noise_texture: adds randomness to the splatter, making it look more natural and less repetitive.

splatter_scale: controls the overall size of the splatter effect.

distortion_amount: adjusts the amount of distortion applied to the splatter, creating a more chaotic look.

edge_variation: adds variation to the edges of the splatter, making it look less uniform.

noise_power: controls the contrast of the noise texture.

Vertex Shader

Like the splash shader, we use ***INSTANCE_ID*** to give each splatter a unique color based on its instance ID. We also use ***INSTANCE_CUSTOM.y*** to add some variation to the green channel, which can be used to control the edge variation of the splatter.

Fragment Shader

Like the splash shader, we calculate the position and angle for each pixel. The difference is that we use ***splatter_scale*** to control the size of the splatter effect.

We also calculate a ***mask*** based on the ***gradient*** and ***texture_value***, which determines how much of the splatter is visible. The ***mask*** is then multiplied by the main color to get the ***final_color*** of the splatter.

The edge fade is calculated using the green channel of the color, which is influenced by ***INSTANCE_CUSTOM.y***. This allows for the splatter to fade over time.

ALPHA is set to the edge fade multiplied by the mask, which sets the transparency of the splatter and ***ALPHA_SCISSOR_THRESHOLD*** is now used based on the mask.

What sets this apart from the splash shader is that it uses a diffuse lighting model and specular highlights, making it interact more with the environment. This shader also has edge fading and a different gradient calculation.